

A quick introduction to ERRA and the ERRA script

James Kirchner 20 March 2022

(version 2, updating version 1 from 28 November 2021)

Here I present a series of vignettes that introduce some of the capabilities of the Ensemble Rainfall-Runoff Analysis (ERRA) script, and show how to invoke them. A bit of hand-holding is provided at the beginning for users who are not very familiar with R.

ERRA is a model-independent, data-driven, nonparametric method for inferring landscapes' nonstationary and nonlinear response to precipitation, using RRDs (Runoff Response Distributions). The mathematics underlying ERRA is explained in Kirchner, "Impulse response functions for nonlinear, nonstationary, and heterogeneous systems, estimated by deconvolution and de-mixing of noisy time series" (2022, in review, hereafter denoted K2022).

The ERRA code is a shell, with a focus on hydrological applications, that runs the IRFnnhs.R script described in that paper. ERRA also adds the capability to set up complex problems exploring rainfall-runoff relationships for precipitation subsets that are split by multiple interacting factors (via the `split_params` option, explained in sections 3 and 5 below). It also provides for aggregation of time steps, as well as automatic calculation of peak height, width, timing, etc. for runoff response distributions.

Preliminaries

Before we start: Windows users will find it advantageous install Microsoft R Open (current version is 4.02 at the time this is written), because this provides direct access to the Intel Math Kernel Library (MKL) Basic Linear Algebra Subroutines (BLAS). This will speed up several of the most time-consuming calculations by factors of 10, 20, 30, or more. No changes must be made to the scripts themselves to access the MKL BLAS; R takes care of that automatically.

These scripts call functions from the following packages, which will need to be installed if they are not already available:

```
data.table
dplyr
Matrix
matrixStats
Rcpp
```

Rcpp also requires a C++ compiler. Windows users can get the GNU C++ compiler by installing Rtools, which can be downloaded from <https://cran.r-project.org/>.

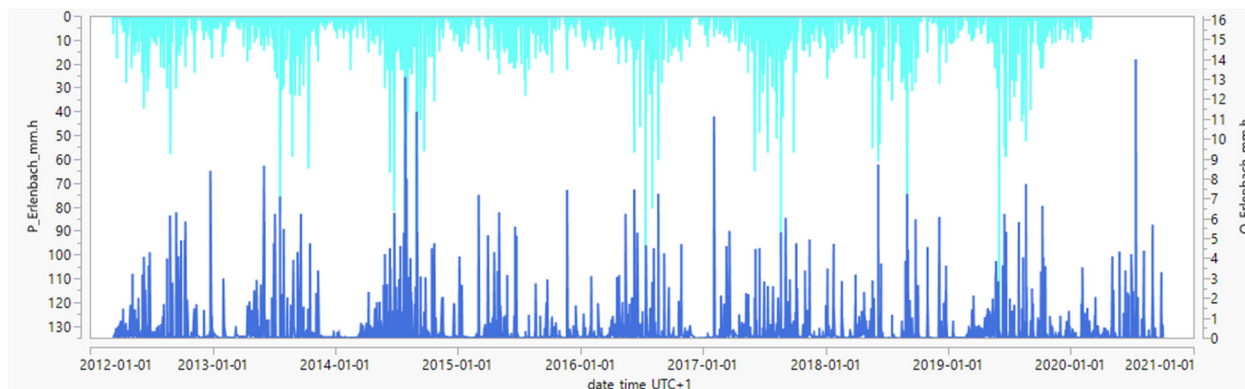
We can get started with the following commands, which will be obvious to any R user:

```
setwd("~/ERRA paper/ERRA vignettes")
rm(list=ls())
source("ERRA_v1.0n.R")
```

The first line sets the working directory – and should be changed to whatever your own working directory is, obviously! The second line clears all existing R objects from memory. The third line loads the ERRA script; this step may take some time, because some functions written in Rcpp need to be compiled and linked in C++.

Test data

The test data for the vignettes presented here is in the file "vignettes_data.txt", which contains roughly 8 years of 10-minute measurements of precipitation, discharge, and weather data from the Erlenbach catchment in Alptal, Switzerland. The precipitation and discharge time series are shown below.



We can read in the test data with:

```
dat <- fread("vignettes_data.txt", header=TRUE, sep="\t",
na.strings=c("NA", ".", "", "#N/A"))
```

We can query what's in the resulting data table with `str(dat)`, which will yield, in this case:

```
Classes 'data.table' and 'data.frame': 450655 obs. of 12 variables:
 $ date_time.UTC+1      : chr "2012-03-07 10:50" "2012-03-07 11:00" "2012-03-07 11:10" "2012-03-07 11:20" ...
 $ hour_ended           : chr "2012-03-07 11:00" "2012-03-07 11:00" "2012-03-07 12:00" "2012-03-07 12:00" ...
 $ year                 : num 2012 2012 2012 2012 2012 ...
 $ water_yr             : int 2012 2012 2012 2012 2012 2012 2012 2012 2012 2012 ...
 $ month               : int 3 3 3 3 3 3 3 3 3 ...
 $ RH_%                : num 72.8 66.1 63.2 63.2 76.8 65.2 62 60.2 62 63 ...
 $ airtemp_degC         : num 0.1 1 1 1 0.5 1.3 1.7 2 1.7 2.1 ...
 $ TAU_dewpoint_degC    : num -4.2 -4.6 -5.2 -5.2 -3.1 -4.5 -4.8 -4.9 -4.8 -4.2 ...
 $ snowdepth_cm         : num 97 97.2 97.6 98.2 98.1 97.1 96.2 97.1 96.5 96 ...
 $ groundwater_cm_below_surface: num 17.9 17.4 17.9 17.9 17 17 17 17 17 17.9 ...
 $ P_Erlenbach_mm.h     : num 0 0 0 0 0 0 0 0 0 ...
 $ Q_Erlenbach_mm.h     : num 0.0725 0.0725 0.0725 0.0725 0.0715 ...
```

Now let's copy precipitation and discharge into vectors with simpler names:

```
p <- dat$P_Erlenbach_mm.h
q <- dat$Q_Erlenbach_mm.h
```

Let's also define a boolean vector that we can use to filter out snowy seasons from the record. At Erlenbach, snow is often present from November through May, so:

```
snowfree <- ((dat$month>5)&(dat$month<11))
```

Most users will only need to call one function, `ERRA`; this will, in turn, call other stuff "under the hood". A complete list of `ERRA` options and what they do, and a description of all the outputs, is found in 200+ lines of comment blocks directly below the `ERRA` function declaration. Some

important notes about performance are also included in a comment block at the top of the script. These comment blocks will not be repeated here, and users are strongly encouraged to read them.

1. First simple analysis

As a first analysis we can try

```
zz <- ERRA(p=p, q=q, m=144, Qfilter=snowfree, xknots=NULL, dt=1/6)
```

where `m=144` tells `ERRA` that we want to analyze lags of up to 144 time steps (i.e., 24 hours), `Qfilter=snowfree` tells `ERRA` to ignore all discharge time steps outside the snow-free months, and `xknots=NULL` tells `ERRA` to ignore the nonlinear dependence of the results on precipitation intensity; more about this in section 6 below.

The option `dt=1/6` tells `ERRA` that we want our results in time units for which the data set's time step is $1/6$ (meaning, in this case, that we have 10-minute data but we want our results in units of hours; the data set's time step is $1/6$ of an hour, so `dt=1/6`). If we had hourly data but wanted our results in units of days, `dt` would be $1/24$; if we had daily data but wanted our results in units of hours, `dt` would be 24. By default `dt=1`, which means that the results are in units of whatever time step the underlying data are. Note that this affects the units in which the results are reported (and thus their numerical values), but not the time intervals at which the analysis is performed or results are reported, which are determined by the sampling interval of the underlying data, potentially as aggregated by the `agg` parameter (see section 4 below for details).

I belabor this point because Runoff Response Distributions (RRDs) are, as the name implies, distributions (fractions per unit of time), and thus their numerical values are inherently dependent on their units. Specifying `dt=1/6` means that our RRDs will be reported in fractions per hour, which is more intuitive than fractions per 10 minutes. This means that the numerical values will also be 6 times greater than if the results were reported in fractions per 10 minutes (but also 24 times smaller than if they were reported in fractions per day, and so on).

The call to `ERRA` shown above puts the results of our analysis into the R `list` data structure `zz`. We can write some key results into tab-delimited text files for further viewing and plotting as follows:

```
fileID <- "Erl_10m_simple_"
with(zz, {
  fwrite(RRD, paste0(fileID, "RRD.txt"), sep="\t")
  fwrite(peakstats, paste0(fileID, "peakstats.txt"), sep="\t")
  fwrite(Qcomp, paste0(fileID, "Qcomp.txt"), sep="\t")
})
```

Verbose reporting is invoked by default, so when we ran `ERRA` using the command above, it unspooled a series of progress reports and timings:

```
initialization done...: 0.154
at h= 0 and time= 3.679 residual PACF (first 5 terms) = 0.9699 -0.3955 0.0505 0.0071 0.005
at h= 1 and time= 6.88 residual PACF (first 5 terms) = 0.3905 -0.0857 -0.0269 -0.0305 -0.0305
at h= 2 and time= 9.924 residual PACF (first 5 terms) = 0.0244 -0.0475 -0.0167 -0.0076 -0.0103
practical (in)significance test passed at h=2
IRF finished...: 9.926
```

```

minimum column count of nonzero x values = 18168 (99% of maximum count, and 10.3% of rows with nonzero weight)
AR coefficients (phi) = 1.360521 -0.4021367
residual PACF (first 5 terms) = 0.0244 -0.0475 -0.0167 -0.0076 -0.0103
return from IRF...: 10.098
ERRA_finished...: 10.371

```

(Your timings will of course be different!)

Setting `verbose=FALSE` will suppress all this chatter, allowing only error and warning messages to get through. But let's take a look at what's here, and what it reveals about what the `ERRA` routine is doing.

The first five lines show `ERRA` iteratively trying successively higher orders of autoregressive noise correction, and the first five terms of the partial autocorrelation function of the residuals from each attempt. For ideal white noise – which we almost never find in the real world – these partial autocorrelations would all be zero. The parameter `h` can be used to specify a particular order of autoregressive noise correction, but the default `h=NULL` tells `ERRA` to automatically search for an appropriate level of noise correction. This search ends when the residual autocorrelation and partial autocorrelation become statistically insignificant, or when they become practically insignificant (i.e., too small to care about), as specified by the parameters `ARprob` and `ARlim`. In this case, that happens at `h=2`.

The line "*minimum column count of nonzero x values = 18168, (99% of maximum, and 10% of total rows with nonzero weight)*" can be important (though not so important in this case). This number, 18168, is the minimum count of nonzero precipitation values at any lag. If the precipitation series is split among multiple columns (to examine nonstationary or nonlinear behavior, for example), this number will be the minimum count of nonzero precipitation values at any lag in any column. The two percentages show that this number is 99% of the largest count of nonzero values in any column (because different numbers of nonzero values will be available at each lag), and only 10% of the total number of rows (because during most of the 10-minute time intervals, no rain falls). In our example, we have started with a data set with nearly half a million rows (450655, to be exact), but over half of these have been excluded because they fall within months that could have snow, and many of the remaining time steps are rainless.

If your RRDs seem strange, and particularly if they are sensitive to small differences in the input data, check these numbers. It is possible that because of the way in which you have split the precipitation time series, one of the subsets has very few nonzero values, and thus one or more of the coefficients cannot be reliably estimated.

The fitted autoregression coefficients are shown in the line "*AR coefficients (phi) = 1.360521 -0.4021367*". These coefficients are large, have opposite signs, and sum to almost exactly 1, suggesting that the **modeled noise is nearly perfectly autocorrelated**, but also that this is not first-order autocorrelation but rather a combination of autoregressive and moving-average behavior. You can also see this in the line "*at h= 0 and time= 3.679 residual PACF (first 5 terms) = 0.9699 -0.3955 0.0505 0.0071 0.005*", which gives the first 5 terms of the partial autocorrelation function (PACF) of the residuals before any ARMA correction is applied. The phi coefficients correct for this ARMA noise as if it were pure AR noise (which will always be possible, if you include enough terms). Note that the first phi coefficient is almost exactly equal to the sum of

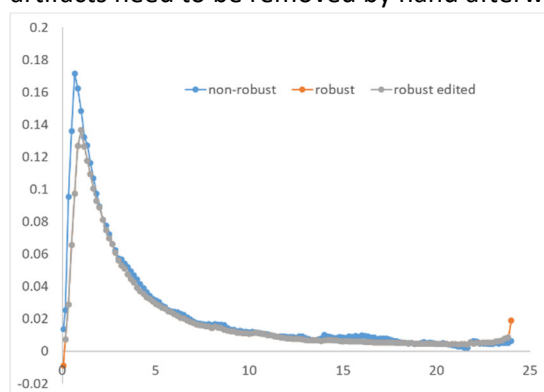
the first two terms of the PACF, and the second phi coefficient is also quite close to the second term of the PACF. The phi coefficients and the PACF express the same information, but in different forms.

The autocorrelation in the residuals could arise from the characteristics of the measurement noise. It could also arise from the fact that the regression model is incomplete: it does not account for variations in either ambient conditions or precipitation intensity, and the largest analyzed lag is 144 time steps (or 24 hours), implying that if there has been no precipitation in the previous 24 hours, discharge must equal a constant. Whatever the origins of the autocorrelation in the residuals, the fitted AR coefficients can effectively whiten them, as shown by the lack of any large partial autocorrelations on the next line: *residual PACF (first 5 terms) = 0.0244 -0.0475 -0.0167 -0.0076 -0.0103*". The whitening of the residuals is important for ensuring that the RRD coefficients and their standard errors are not distorted by residual autocorrelation.

2. Robust vs. non-robust estimation

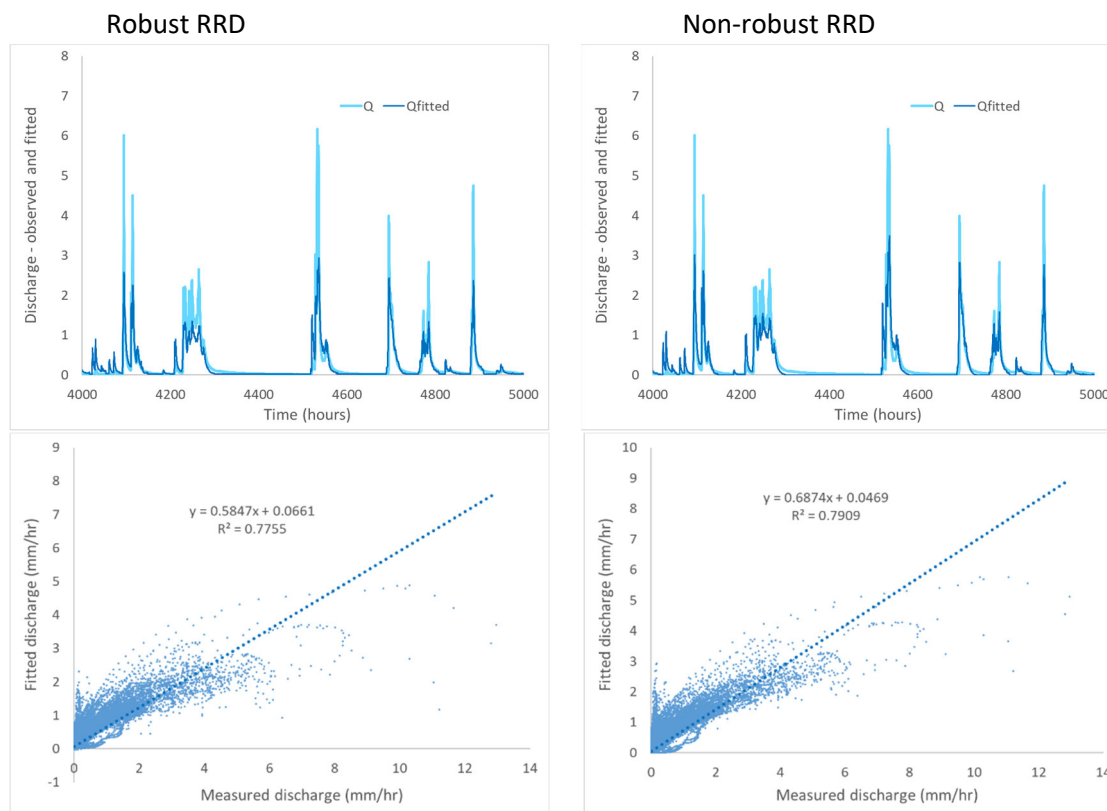
Robust estimation is turned off by default, but can be turned on by setting `robust=TRUE`. Non-robust RRDs will be more sensitive to outliers and data quirks than robust RRDs. RRDs calculated without robust estimation will also give more emphasis to the landscape's response to the largest precipitation events, which have the greatest leverage on the solution (since they lie farthest from the average). By contrast, RRDs calculated with robust estimation will tend to reflect the landscape's response to more typical precipitation events, but will be less likely to over-represent the high flows associated with the highest-intensity precipitation. Later, we will see that a smart way to handle these issues is to explicitly represent the nonlinearity in the system's response to precipitation.

To illustrate the effects of robust estimation, we can re-run the analysis above, but this time with `robust=TRUE`, and compare the results. The plot below compares robust estimation (in orange and gray) and non-robust estimation. Note that the robust RRD has anomalous values for the first and last coefficient (the orange points). These are artifacts of the ARMA noise in the residuals. The effects of ARMA noise are suppressed in non-robust RRDs by the ARMA noise correction procedure. However, ARMA noise correction is incompatible with the robust estimation procedure, so these artifacts need to be removed by hand afterwards.



It is tempting to ask, "which one is correct?", but that misses the point. The two RRDs simply show different things. The robust RRD reflects the response to typical precipitation inputs, whereas the non-robust RRD gives greater emphasis to the largest precipitation inputs, which also generate the largest discharges (and do so a bit quicker, too).

The data table Q_{comp} allows visual comparisons of the observed and fitted discharges, like those shown in the plots below. The top row shows brief excerpts from the time series, and the bottom row shows scatterplots of the observed values vs. the fitted ones. Results for robust estimation are on the left and non-robust on the right.



At this point it is important to emphasize that the objective of ERRA is not to predict hydrographs, but instead to characterize the relationship between precipitation and discharge, and how it varies among landscapes and over time as ambient conditions vary. Thus the plots above are intended mostly as clues to the RRDs' limitations as descriptions of catchment behavior. One can see that both the robust and non-robust RRDs underestimate the discharge peaks. Unsurprisingly, this discrepancy is greater for the robust RRD, because, like any robust statistic, it will be less influenced by the relatively small number of high-precipitation points. One can also see that the scatterplots show hysteresis (at least at the highest flows), suggesting timing errors in estimating the response of streamflow to precipitation.

So far, we have done nothing very different from decades of unit hydrograph studies. But both classical unit hydrographs and the simple RRDs presented above are based on the assumption that the rainfall-runoff relationship is linear and stationary – that is, that a millimeter of precipitation has the same effect on streamflow, no matter whether it falls on wet or dry ground, and no matter whether it falls as drizzle or as an intensive cloudburst. In many landscapes we know that this assumption is false.

Thus the simple RRDs above are trying to infer the average response to (for example) a mixture of wet and dry antecedent conditions, and high-intensity and low-intensity precipitation. The robust and non-robust RRDs average over this behavior in different ways; the robust RRD limits the

leverage of the high-precipitation points, and thus better represents conditions that prevail most of the time, whereas the non-robust RRDs skew that average, emphasizing the high-intensity precipitation that generates the highest flows. But neither of these averages is guaranteed to be a particularly faithful characterization of a nonstationary, nonlinear system. Therefore one of the main points of ERRA is to provide tools for dissecting the rainfall-runoff relationship, mapping out how it varies depending on ambient conditions and rainfall characteristics. How it does this is the main focus of the next several sections.

3. Quantifying nonstationary rainfall-runoff behavior

ERRA allows users to split the precipitation record according to different conditions prevailing at the time that the precipitation falls, and to track the runoff responses under these different conditions, even if they overlap with one another (for example, even if the runoff response to precipitation falling on dry ground overlaps with the runoff response to later precipitation that falls on wetter ground). Users who want to understand the underlying mathematics are referred to K2022; users who want to understand how this works computationally are referred to the scripts themselves. Here we will instead describe how to set up such an analysis and how to interpret the results.

Splitting the precipitation record according to ambient conditions is controlled by the `split_params` option in ERRA. Users tell ERRA how to split the precipitation time series by specifying a list of splitting parameters. Here is an example:

```
lagQ_split <- list(
  crit = list(q) ,
  crit_label = "lagQ" ,
  crit_lag = 6,
  breakpts = list(c(50, 80, 95)) ,
  pct_breakpts = TRUE ,
  thresh = 0 ,
  by_bin = TRUE)
```

`crit` specifies a list of one or more variables that will be used as splitting criteria. This must be specified as a list, even if, as in this case, it consists of just a single vector of data. In this case, we specify the discharge vector `q`, because we will use lagged values of discharge as an indication of antecedent wetness conditions. If we specify multiple criteria, they will be nested in the order specified (see section 5 for an example in which lagged discharge is nested within ambient temperature as a measure of rain/snow precipitation phase).

`crit_label` specifies a short text label for the variable in `crit` (or a vector of such labels, if `crit` contains multiple criterion variables). Here our label is `lagQ`, reflecting lagged discharge.

`crit_lag` is an integer (or vector, if there are multiple criteria) specifying the lag (or, if negative, the lead), in number of time steps to be applied to the variable(s) specified in `crit`. Here we specify a lag of 6 time steps, which equals one hour for our data. Note that these lags are specified in time steps *following* any data aggregation implied by the `agg` parameter in ERRA (an example will be shown in section 4). For example, if we aggregated the data to half-hourly time steps by specifying `agg=3`, setting `crit_lag=6` would create a lag of 6 half-hourly time steps, or three hours.

`breakpts` is a list of vectors, one for each variable in `crit`, specifying the values at which the precipitation record should be split.

`pct_breakpts` is a boolean flag (or a vector of such flags, if there are multiple criteria), indicating whether the breakpoints specified in `breakpts` are in percentiles (`TRUE`) or in the units of the criterion variables themselves (`FALSE`).

`thresh` is a real value (or a vector of such values, if there are multiple criteria), specifying the thresholds above which percentiles will be calculated for the variable(s) specified in `crit`. If `pct_breakpts` is `FALSE` for the variable in question, `thresh` has no effect. But if `pct_breakpts` is `TRUE` (and thus the splitting breakpoints are set by percentiles), those percentiles will only be calculated from values greater than `thresh`. This has the advantage that users do not need to know what fraction of the `crit` distribution is below `thresh`, which may be large in highly skewed distributions (such as antecedent precipitation, where many values may be zero and one wants to specify percentiles of the non-zero values instead). All values below or equal to `thresh` will be included in the lowest bin of `crit`, below the first breakpoint; they just won't be used to convert the percentile breakpoints to their equivalent values of the `crit` variable.

`by_bin` is a boolean flag (or vector of such flags, for multiple criteria), specifying whether, for nested criteria, percentile breakpoints should be calculated separately for each bin that they are nested into. Why would this be useful? Consider a case where we want to split precipitation according to cold vs. warm conditions, and according to antecedent discharge within both cold and warm conditions (see section 5). If `by_bin=FALSE`, then (say) a breakpoint of 95 percent would be set according to the 95th percentile of antecedent discharge over both warm and cold conditions combined. But if this breakpoint value exceeds all of the antecedent discharges under cold conditions, the analysis will fail since the above-95-percent bin under cold conditions will have nothing in it. Setting `by_bin=TRUE` avoids this problem by calculating the breakpoint percentiles separately for each nested bin (separately for warm and cold conditions, in our example). `by_bin` has no effect if `pct_breakpts` is `FALSE` for the corresponding criterion variable, but a value of `TRUE` or `FALSE` must be specified nonetheless. `by_bin` also has no effect on the first criterion variable (which is not nested within any other criterion), but again a value of `TRUE` or `FALSE` must be specified nonetheless.

Breakpoint values must not lie outside the range of values of the corresponding criterion variable (or subsets of those values, for criteria that are nested inside other criteria). This will trigger an error message and a hard stop. This problem can be avoided by specifying breakpoints as percentiles, particularly for criteria that are nested inside of other criteria, and specifying `by_bin=TRUE` for those criteria (this has the drawback that the numerical values of the breakpoints may vary from bin to bin, however).

The breakpoints must also not result in bins that contain no non-zero precipitation values, because this will lead to columns of zeroes in the design matrix, and thus to singularities in the crossproducts matrix. If you get a message like *"Error in solve.default(C, xy): Lapack routine dgesv: system is exactly singular: U[146,146] = 0"*, this may be the cause. In this example it is column 146 of the design matrix that is the problem, so that will give you a clue where to look.

Singularity errors can also arise (don't ask me how I know...!) if `pct_breakpts=FALSE` but the elements in `breakpts` are actually specified in percentages (or vice versa), or if `pct_breakpts=TRUE`, but the breakpoints are specified as fractions (e.g., 0.5, 0.8, 0.9, 0.99) rather than percentages (e.g., 50, 80, 90, 99).

Whew! I know it may seem complicated, but these parameters allow one to specify many different strategies for splitting the precipitation record to quantify catchment runoff response under different ambient conditions. So let's see some examples of what we can do. Let's run this:

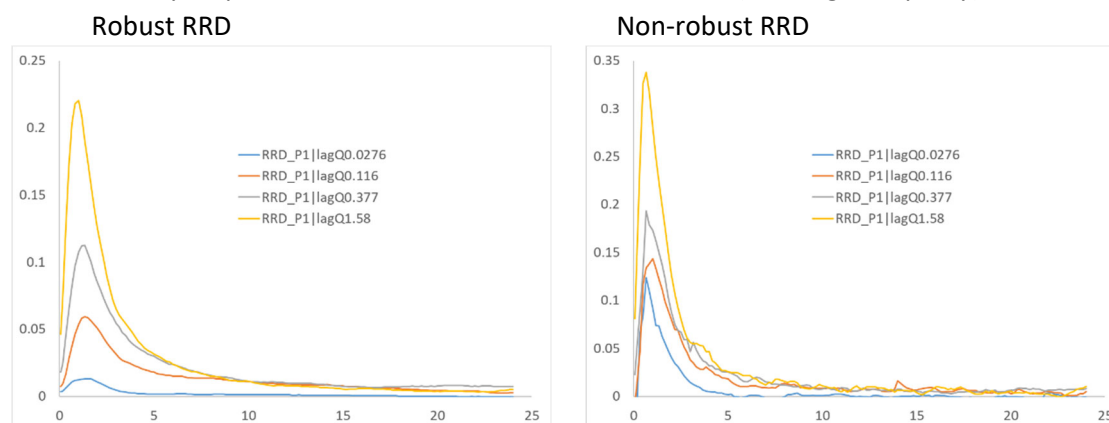
```
zz <- ERRA(p=p, q=q, m=144, split_params=lagQ_split, knots=NULL,
Qfilter=snowfree, dt=1/6)
```

or, for robust RRDs:

```
zz <- ERRA(p=p, q=q, m=144, split_params=lagQ_split, knots=NULL,
robust=TRUE, Qfilter=snowfree, dt=1/6)
```

The plots below compare the results of these two calculations:

Nonstationary response to variations in antecedent wetness (with lag Q as proxy)



First of all, note the legends for the individual curves, which are identical to the column headings in the RRD data table. "RRD_P1" means that this is the first precipitation column (this will often be the only precipitation column, but ERRA can also treat cases with multiple precipitation sources. "lagQ0.0276" means that the average lagQ in the lowest bin (the lowest 50 percent of lagQ, given the breakpoints specified above) is 0.0276, and "lagQ1.58" means that the average lagQ in the highest bin (the highest 5% of lagQ) is 1.58.

The robust RRDs shown in the left-hand plot show the substantial effects of differences in antecedent wetness. These are less evident in the non-robust RRDs shown in the right-hand plot, because a few large rain events give a substantial runoff response even if the catchment starts out relatively dry, so the blue curve on the right looks qualitatively similar to the other curves on the right, but qualitatively different from the blue curve on the left.

It is also visually obvious that the robust RRDs are smoother than the non-robust RRDs. This does not result from any explicit smoothing by the robust estimation algorithm; instead, it results from

the robust algorithm's insensitivity to individual points with large residuals, which generate a lot of the noise in the non-robust RRDs.

4. Aggregating time steps

Quantifying runoff response over long lag times requires many time steps if the data sampling frequency is high. At the 10-minute sampling frequency of our data set, for example, measuring runoff response at lags of 1 day requires 144 lag periods in the RRD, and lags of 1 week would require 1008 lag periods. The width of the design matrix will scale as the number of lag periods, times the number of different subsets being analyzed, times the number of knots if one wants to assess the nonlinearity of the system response (more about this below). The order of difficulty of the matrix calculations in ERRA scales as the square, or as an even higher power, of the width of the design matrix. Thus having so many lag periods can multiply the computational burden of these calculations dramatically.

One way to solve this problem is to aggregate the underlying data into coarser time steps. This is what the `agg` parameter does: it averages the source data over sets of `agg` consecutive time steps. Thus to aggregate the 10-minute data to hourly intervals, set `agg=6`, to aggregate hourly data to daily time steps, set `agg=24`, and so forth.

Aggregating the input data can also help in clarifying the nonlinear response to precipitation intensity. If (say) the characteristic response time of the catchment is on the order of hours, we would not expect that minute-to-minute variations in precipitation intensity will have a detectable effect on runoff; instead we would expect runoff to exhibit clearer responses to precipitation intensity variations that are averaged over timescales comparable to the catchment's response time.

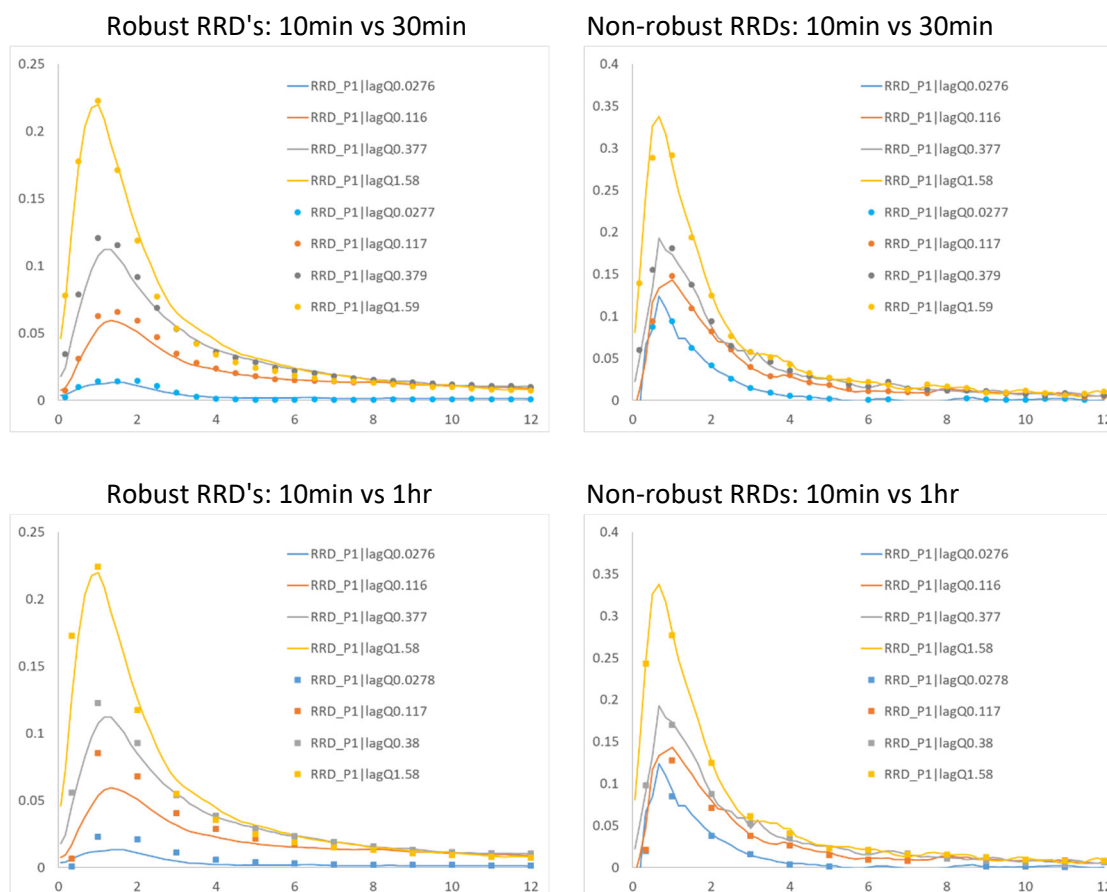
Note that using `agg` to aggregate time steps will also affect anything that is based on numbers of time steps. For example, if we aggregate 10-minute data to hours using `agg=6`, but keep `m` at 144, then we will be calculating RRDs at lags up to 144 hours instead of 144 10-minute intervals. Likewise if we aggregate 10-minute data to hourly data but keep `crit_lag=6` for `lagQ` in our list of splitting parameters, we will now be splitting according to hourly averaged `Q`, lagged by 6 hours (instead of 10-minute `Q`, lagged by one hour). Therefore we can see how aggregation to 30 minutes affects the results of the analysis presented in the previous section – but keep the lag of discharge at one hour – with the following instructions to ERRA:

```
lagQ_split <- list(
  crit = list(q) ,
  crit_label = "lagQ" ,
  crit_lag = 2,
  breakpts = list(c(50, 80, 95)) ,
  pct_breakpts = TRUE ,
  thresh = 0 ,
  by_bin = TRUE
)

zz <- ERRA(p=p, q=q, m=48, split_params=lagQ_split, knots=NULL, agg=3,
Qfilter=snowfree, dt=1/6)
```

```
zz <- ERRA(p=p, q=q, m=48, split_params=lagQ_split, knots=NULL,
robust=TRUE, agg=3, Qfilter=snowfree, dt=1/6)
```

The figures below show how aggregating our 10-minute data to 30 minutes and to 1 hour affects the fitted RRDs. The 10-minute RRDs are shown by the lines and the 30-minute and 1-hour RRDs are shown by the points. The aggregated RRDs closely approximate the higher-resolution RRDs, although obviously also with some loss of information about how to "connect the dots" between the lower-resolution RRD estimates.



5. Splitting by multiple criteria

What if we want to analyze how runoff response to precipitation varies depending on both the air temperature when the rain falls, *and* the antecedent wetness (as proxied by the antecedent discharge an hour beforehand)? The `ERRA` script can do this; in fact it can split the precipitation time steps according to an unlimited number of nested criteria (unlimited, that is, subject to the availability of sufficient data). The example script below first splits the precipitation records according to the ambient temperature (T), and then by $\text{lag}Q$ within each temperature bin. The line `crit_lag = c(0, 2)` says that temperature will not be lagged, but $\text{lag}Q$ will lag q by two time steps (which will be 1 hour, because we are setting `agg=3` in the call to `ERRA`). The specifications `breakpts = list(c(0, 5), c(50, 80, 95))` and `pct_breakpts = c(FALSE, TRUE)` jointly tell `ERRA` that we want to split T at breakpoints of 0 and 5 degrees (not percentages!),

meaning that we have three temperature bins (colder than 0, 0 to 5, and warmer than 5), and that within each of these temperature bins, we want to split antecedent wetness into four percentile bins (below the median, 50th-80th percentile, 80th-95th percentile, and above the 95th percentile). The specification `by_bin = c(FALSE, TRUE)` tells `ERRA` that we want to evaluate these `lagQ` percentiles separately within each temperature bin (because the distribution of antecedent discharge could be quite different under sub-freezing conditions).

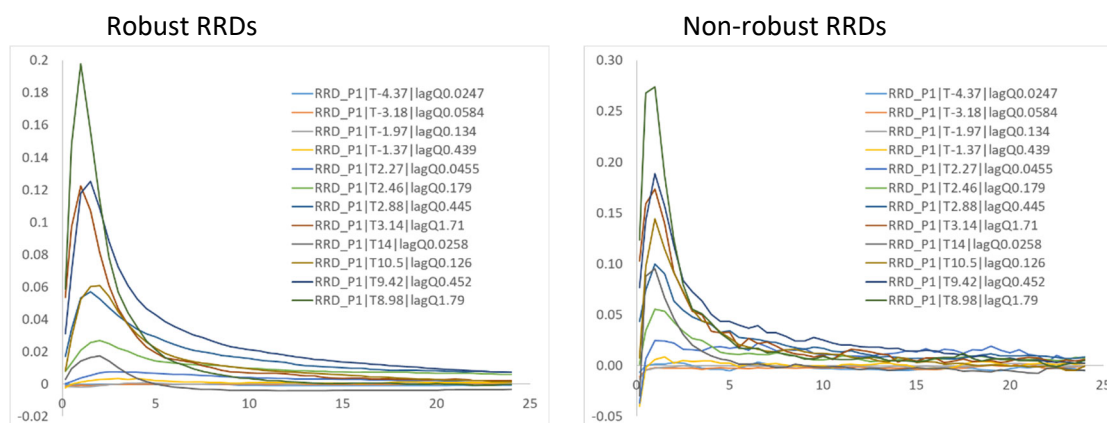
```
p <- dat$P_Erlenbach_mm.h
q <- dat$Q_Erlenbach_mm.h
T <- dat$airtemp_degC

airtemp_lagQ_split <- list(
  crit = list(T, q) ,
  crit_label = c("T", "lagQ") ,
  crit_lag = c(0, 2),
  breakpts = list(c(0, 5), c(50, 80, 95)) ,
  pct_breakpts = c(FALSE, TRUE) ,
  thresh = c(-999, 0) ,
  by_bin = c(FALSE, TRUE)
)

zz <- ERRA(p=p, q=q, m=48, split_params=airtemp_lagQ_split, knots=NULL,
robust=TRUE, agg=3, dt=1/6)

fileID <- "Erl_30m_airtemp_lagQ_robust_"
with(zz, {
  fwrite(RRD, paste0(fileID, "RRD.txt"), sep="\t")
  fwrite(peakstats, paste0(fileID, "peakstats.txt"), sep="\t")
  fwrite(Qcomp, paste0(fileID, "Qcomp.txt"), sep="\t")
})
```

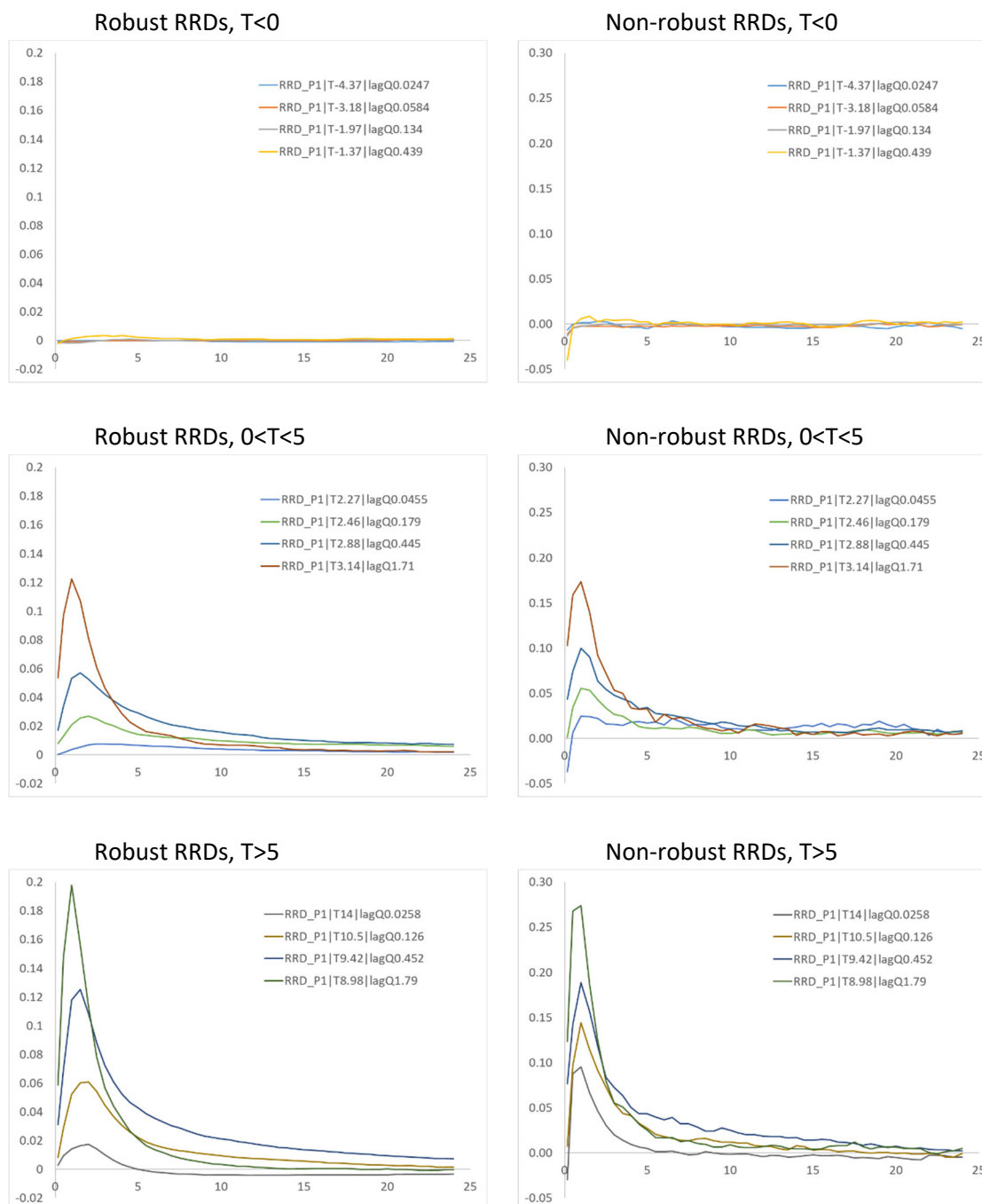
This call to `ERRA` generates a set of 12 RRDs, as shown in the figures below. First we see all the RRDs together. Note that each column label shows the average values of both `T` and `lagQ` that correspond to each bin (instead of just `lagQ`, as in section 3 above).



Note that in some cases, the non-robust RRDs show substantial negative values at lag zero. The robust RRDs do not show such large artifacts, suggesting that the robust estimation approach may allow relatively weak signals (such as runoff responses under cold or dry conditions) to be estimated

reliably in the presence of much stronger signals (such as runoff responses under warm and wet conditions).

Below we see the same curves as those shown above, on the same axes, separated into low, intermediate, and high temperatures. In each plot there are four curves, representing four percentile ranges of antecedent discharge.



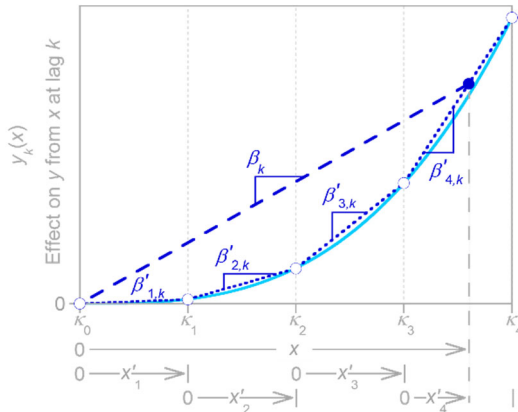
This is a good place to introduce another output from the ERRA script, the `peakstats` table, which summarizes the RRDs from analyses like this one. As shown below, the first few columns of the `peakstats` table show a text label and the splitting criteria (lower bound, upper bound, and mean) for each precipitation subset. Then the peak time `tpeak` and the peak height `peakht` follow, both

estimated from a quadratic fit to the uppermost 20% of each RRD (including at least the highest RRD value and the two adjacent ones). These values are blank if no peak can be identified (e.g., if the last lag has the highest RRD coefficient, or all RRD values are less than zero). The `peakstats` table also shows the lag time and the value of the single highest RRD coefficient, labeled `alt_tpeak` and `alt_peakht`, respectively, which are provided mostly as a sanity check for `tpeak` and `peakht`. Then the `peakstats` table gives the width of the peak at half of `peakht`, as estimated from linear fits to all points on each side of the peak that have values between 40% and 60% of `peakht` (including at least the two points bracketing 0.5 times `peakht`). `Peakstats` also provides an estimate of the runoff coefficient `rc`, which is the integral of the RRD over the range of lag times that have been analyzed. The last column of `peakstats` is `n.nz`, the number of nonzero precipitation values in the precipitation subset that is represented by the corresponding row of the `peakstats` table.

set_label	lwr_T	lwr_lagQ	upr_T	upr_lagQ	mean_T	mean_lagQ	tpeak	tpeak_se	peakht	peakht_se	alt_tpeak	alt_peakht	alt_peakht_se	width	width_se	rc	rc_se	n.nz
P1 T-4.37 lagQ0.0247	-21.1333	0.000986	0	0.039945	-4.36696	0.0246561										-0.017	0.00376	2234
P1 T-3.18 lagQ0.0584	-21.1333	0.039945	0	0.089277	-3.17753	0.0583595										-0.005	0.005165	1791
P1 T-1.97 lagQ0.134	-21.1333	0.089277	0	0.227836	-1.97168	0.1343104	4.319248	1.060074	0.000713	0.000686	4.5	0.0007413	0.000865239	1.923	1.754433	0.003	0.005937	1227
P1 T-1.37 lagQ0.439	-21.1333	0.227836	0	0.38548	-1.36693	0.439178	3.222409	0.687562	0.003268	0.000611	3	0.0033679	0.000934971	4.525	1.048534	0.028	0.005988	803
P1 T2.27 lagQ0.0455	0	0.000986	5	0.104055	2.266068	0.0455426	3.600968	0.343731	0.007434	0.000466	2.5	0.0076418	0.000670147	9.494	1.030849	0.092	0.005431	1927
P1 T2.46 lagQ0.179	0	0.104055	5	0.289151	2.460715	0.1787429	2.133239	0.098951	0.026158	0.000889	2	0.0268655	0.000747152	5.653	0.714734	0.263	0.006289	1934
P1 T2.88 lagQ0.445	0	0.289151	5	0.782795	2.87546	0.4445709	1.794338	0.092848	0.055913	0.002742	1.5	0.056949	0.000874449	4.764	0.292538	0.449	0.007275	1560
P1 T3.14 lagQ1.71	0	0.782795	5	10.93578	3.135623	1.7084056	1.128453	0.065995	0.118677	0.005214	1	0.1222423	0.000962218	2.27	0.105095	0.431	0.005595	1188
P1 T14 lagQ0.0258	5	0.000986	31.93333	0.058685	13.97803	0.0257922	1.768077	0.029104	0.017182	0.000368	2	0.0175185	0.000397015	2.623	0.083833	-0.021	0.003994	2278
P1 T10.5 lagQ0.126	5	0.058685	31.93333	0.246904	10.51497	0.1260946	1.903358	0.045436	0.062157	0.001732	2	0.0608403	0.000655718	3.51	0.110273	0.339	0.005922	2562
P1 T9.42 lagQ0.452	5	0.246904	31.93333	0.871562	9.417783	0.4517049	1.560168	0.058718	0.12502	0.005847	1.5	0.1250912	0.000766438	3.111	0.166316	0.714	0.006861	2399
P1 T8.98 lagQ1.79	5	0.871562	31.93333	12.47458	8.980288	1.7892819	1.022326	0.002809	0.198006	0.000723	1	0.1979141	0.00075115	1.901	0.02662	0.501	0.00445	2401

6. Quantifying runoff's nonlinear dependence on precipitation intensity

Real-world hydrologic systems are not just nonstationary (meaning the same rainfall can generate different responses, depending on ambient conditions); they are also nonlinear (meaning the runoff response may not be proportional to the rainfall input). ERRA can quantify the nonlinear response of runoff to rainfall, and can also quantify how the RRD changes in shape, not just in amplitude, between high- and low-intensity precipitation. It does this by replacing each coefficient, at each lag in the RRD, with a piecewise linear "broken-stick" function as shown in Fig. 8 of K2022, reproduced below:



Broken-stick approximation to nonlinear impulse response. Schematic of the (unknown) nonlinear dependence of output y on input x at some lag k , and its approximation by a piecewise linear function defined by breakpoints x_ℓ and the associated slopes $\beta'_{\ell,k}$ of each segment between successive knots. The increments x'_ℓ of x within each segment, as illustrated below the x axis, facilitate the estimation of the impulse response coefficient β_k via broken-stick regression, as described in Eqs (42-47) of K2022. For any given value of x , such as the value indicated by the dashed gray line, the system response is approximated by the corresponding point on the piecewise linear curve (the solid blue circle). This response equals the integral over the piecewise linear curve, that is, the product of the vector of increments x'_ℓ and the vector of the associated slopes $\beta'_{\ell,k}$. The impulse response coefficient β_k is this integral divided by x (i.e., the slope of the dashed blue line). The breakpoints $x = x_\ell$ are specified by the user. The corresponding segment slopes $\beta'_{\ell,k}$, determined by fitting Eq (45) of K2022 to time series of x and y , can be used to infer the associated values of $y_k(x = x_\ell)$ and thus to estimate the nonlinear dependence of y on x at each lag k .

This broken-stick function approximates the relationship between precipitation intensity (x in the above figure) and runoff response (y in the above figure) with a continuous function that is a series of linear segments connecting a set of "knot points" (the open circles of the figure). The lowermost and uppermost knots are set by the algorithm. The lowermost knot is at $x=0$, under the logical assumption that if no rain falls, it will have no effect on current or future runoff (note that this is not the same as saying that current or future runoff will be zero). The uppermost knot is automatically set at the maximum precipitation value in the data set (this is required by the mathematics underlying the broken-stick approach). In between these two limits, the remaining knots are set by the user with the `knots` option in `ERRA`. For example, `knots = c(80, 90, 95, 99)` and `pct_knots=TRUE` will tell `ERRA` to set the knots at the 80th, 90th, 95th, and 99th percentiles of the non-zero precipitation values in the data set.

Note that a nonlinear function like the one shown in the figure above will be estimated separately for every lag in the RRD. From the resulting fitted parameters, users can examine RRD's for different precipitation intensities (i.e., runoff response vs. lag time, for fixed values of precipitation intensity), and can also examine runoff response vs. precipitation intensity, for fixed values of lag time.

We can calculate nonlinear RRDs for the 30-minute snow-free Erlenbach data using the following calls to `ERRA`:

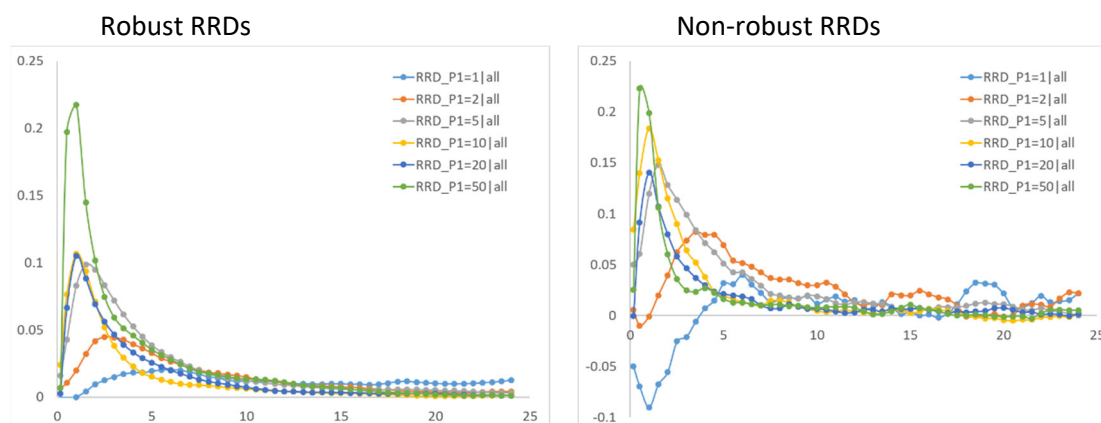
```
zz <- ERRA(p=p, q=q, m=48, Qfilter=snowfree, knots=c(1, 2, 5, 10, 20, 50),
pct_knots=FALSE, agg=3, dt=1/6)
```

and, for robust estimation,

```
zz <- ERRA(p=p, q=q, m=48, Qfilter=snowfree, knots=c(1, 2, 5, 10, 20, 50),
pct_knots=FALSE, robust=TRUE, agg=3, dt=1/6)
```

Here we have set `pct_knots` to `FALSE`, so the knot values are specified in the same units as p (mm/h) instead of in percentiles of p . We have done this to make the results easier to interpret.

The plots below show the RRDs for the different precipitation intensities specified by the knots; I have omitted the maximum p , 62.8 mm/h, because it is poorly constrained by the available data.



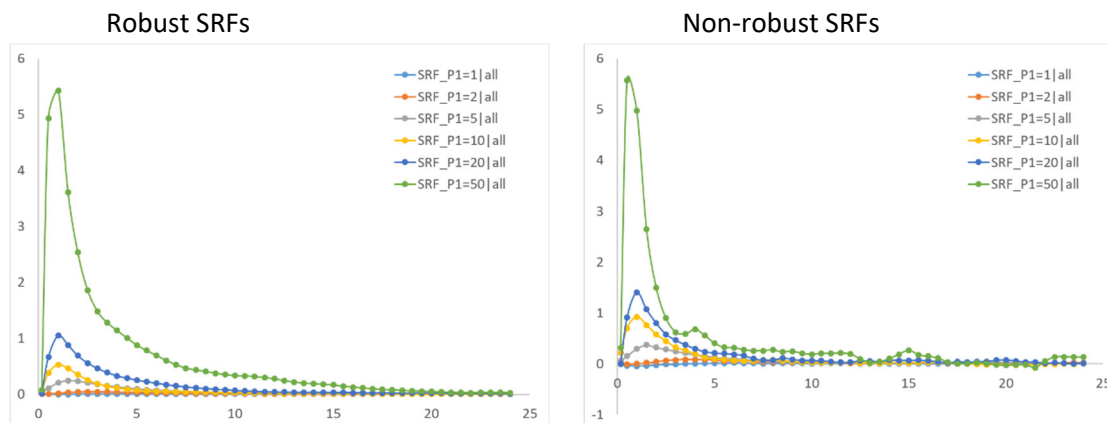
One thing to notice is the format of the column headings (and thus of the figure legends). For example, `RRD_P1=20|all` means:

- that this is the RRD of precipitation variable 1 (this will only be relevant in cases where we have multiple precipitation inputs),
- the knot value is 20, that is, this RRD expresses the runoff response that we should expect when the precipitation intensity is 20 (in whatever units precipitation is measured in), averaged over time steps of length `agg*dt`,
- and we have analyzed all the precipitation, meaning that we have not split the precipitation according to things like antecedent wetness (but note that this does not mention any filtering of the data according to discharge time, such as our elimination of snowy months by specifying `Qfilter=snowfree`).

The broken-stick model illustrated above yields RRD estimates *at the knots*, not over an interval of precipitation intensity. This contrasts with the binning procedure that is used to analyze nonstationary rainfall-runoff relationships, in which the RRD is *averaged* over all the precipitation events that fall into each bin between each pair of breakpoints. The average RRDs for those bins may be skewed toward the higher-intensity precipitation values or the more common precipitation values within those bins, depending on whether `robust=FALSE` or `robust=TRUE`.

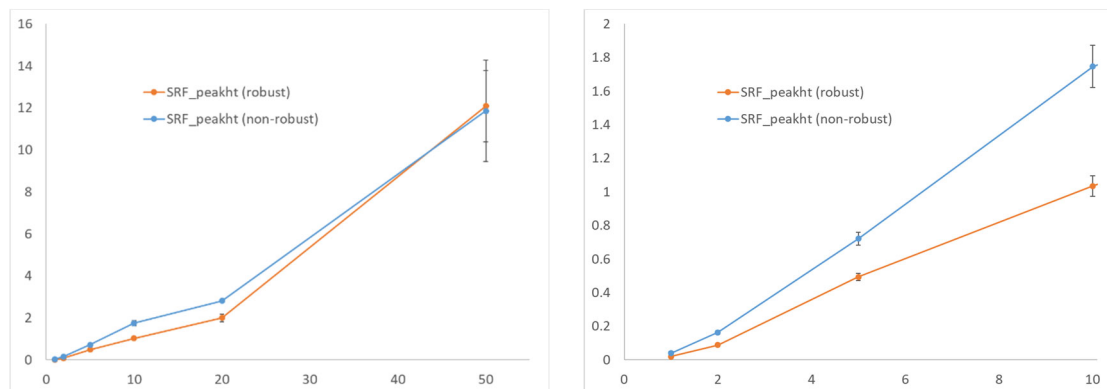
One can see that the robust RRDs are smoother than the non-robust RRDs (because, as mentioned in section 3 above, the robust estimation algorithm is insensitive to individual points with large residuals). One can also see that the non-robust RRDs for weak precipitation inputs show implausible behavior (including large negative values). This apparently arises from interactions between these weak precipitation signals and much stronger signals associated with much larger precipitation inputs. The broken-stick approach, as outlined in section 5 of K2022, is designed to reduce this interference as much as possible (compared with, for example, an approach in which precipitation is simply filtered into different bins), but it cannot completely eliminate it. If high-intensity precipitation inputs are not accurately measured (which is virtually impossible because rain gauges are many orders of magnitude smaller than the catchments that they represent), they will result in large residuals. The non-robust estimation procedure tries to reduce these large residuals, potentially by making large changes to the RRD coefficients of the weaker precipitation inputs. By contrast, the robust estimation algorithm downweights these large residuals, reducing their influence on the RRDs of the weaker inputs.

When it performs nonlinear analyses, `ERRA` generates several additional outputs worth mentioning. One is the Scaled Response Function SRF, which equals the RRD for each knot, multiplied by the value of precipitation at that knot. Whereas the RRD describes the distribution of runoff that is expected *per unit precipitation* that falls at a rate specified by the value of the knot, the SRF rescales the RRD to reflect the *total runoff* that is expected to result from precipitation falling at a rate specified by the knot value. The SRF more clearly reflects the nonlinearity in the runoff response. For example, in a linear system, the RRD will be constant, but the SRF will increase linearly with precipitation intensity. Similarly, in a quadratic nonlinear system, the RRD will increase linearly with precipitation intensity, and the SRF will increase quadratically. The figures below show the robust and non-robust SRFs corresponding to the RRDs in the figures above. These figures make it visually obvious how much more runoff response can be expected from high-intensity precipitation than from low-intensity precipitation.



When it performs nonlinear analyses, `ERRA` appends peak heights for the SRFs to the `peakstats` table (their peak times and widths remain identical to those of the RRDs). It also appends estimates of the runoff volume `rvol`, which is the runoff coefficient `rc` scaled by the precipitation intensity and reflects the total runoff volume one would expect (during the time period covered by the RRD and SRF) as the result of `knot` units of precipitation.

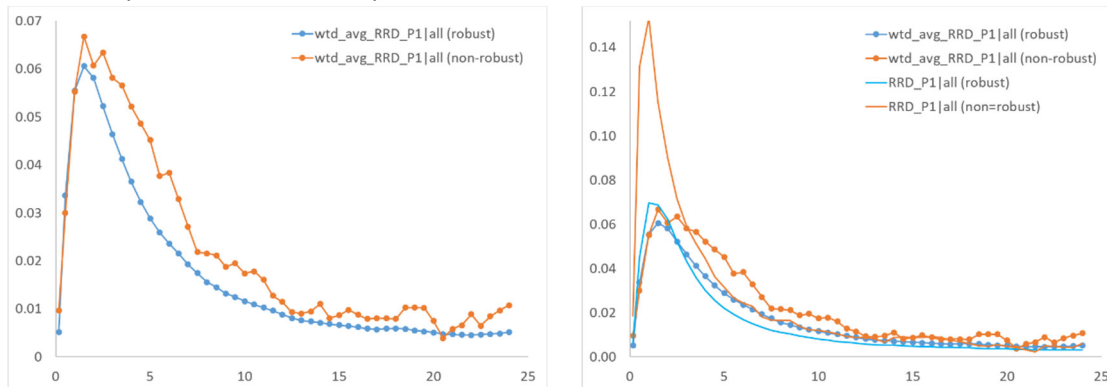
The figures below show the peak height of the SRF, as a function of the precipitation rate at each knot. The right-hand plot shows rates less than 10 mm/h, so that the detail there is easier to see. The overall pattern looks roughly linear, but with an offset of about 1 mm/h below which there is practically no runoff response. This may reflect losses due to canopy interception and surface evaporation.



To estimate the average effect of a unit of precipitation in a nonlinear system, `ERRA` reports the precipitation-weighted average RRD (`wtd_avg_RRD`), calculated via equation (48) of K2022. The precipitation-weighted average RRD is calculated by using the fitted broken-stick curve to calculate the RRD corresponding to the individual precipitation values for each time step and lag. This approach thus accounts for both the intensity-frequency distribution of precipitation rates and the nonlinearity of the landscape's response to precipitation. This approach is necessary because, as demonstrated in section 5.3 of K2022, substantial biases can result from applying the methods of sections 1-5 above to nonlinear systems; that is, those methods do not "average out" the nonlinearities.

The figure on the left below compares the weighted average RRDs calculated by the robust and non-robust methods. One can see that the two curves broadly agree, although they differ in detail. The

figure on the right compares these two weighted average RRDs with the simple RRDs calculated using the methods of sections 1 and 2, which ignore the system's nonlinearity. The robust RRD (blue curve without dots) generally agrees with the robust volume-weighted RRD, but the non-robust RRD (orange curve without dots) deviates substantially from both the robust and non-robust volume-weighted RRDs. Thus it appears that the non-robust RRD yields a significantly biased view of the runoff response of the landscape.



Like all of the results presented here, this will need to be verified with benchmark tests. Assuming that this result holds, however, it would indicate that even if one does not want to explore the nonlinearity in the system's response, accounting for that nonlinearity can be essential for accurately quantifying the system's average response, particularly if one is not using the robust estimation algorithm.

The `ERRA` script also reports the peak height, width, etc. of the precipitation-averaged RRD in the data table `avgRRDpeakstats`. This can be particularly helpful in quantifying how changes in ambient conditions (such as antecedent wetness) affect runoff response in nonlinear systems, if the focus of attention is not on the nonlinearity itself but rather on the average system response.

7. Quantifying rainfall-runoff behavior that is both nonlinear and nonstationary

In interpreting both nonstationary analyses (as in sections 3-5 above) and nonlinear analyses (as in section 6 above), users should keep in mind that precipitation intensity and ambient catchment conditions could potentially be correlated with one another. For example, the same cold or dry weather conditions that lead to low antecedent wetness might also reduce the likelihood of intense precipitation, leading to distorted estimates of how both antecedent wetness and low-intensity precipitation affect the runoff response. One way to sort out the relative importance of antecedent wetness and precipitation intensity is to jointly analyze both factors, thus combining the nonstationarity and nonlinearity approaches outlined in sections 3-6 above. An example call to `ERRA` that performs such an analysis is:

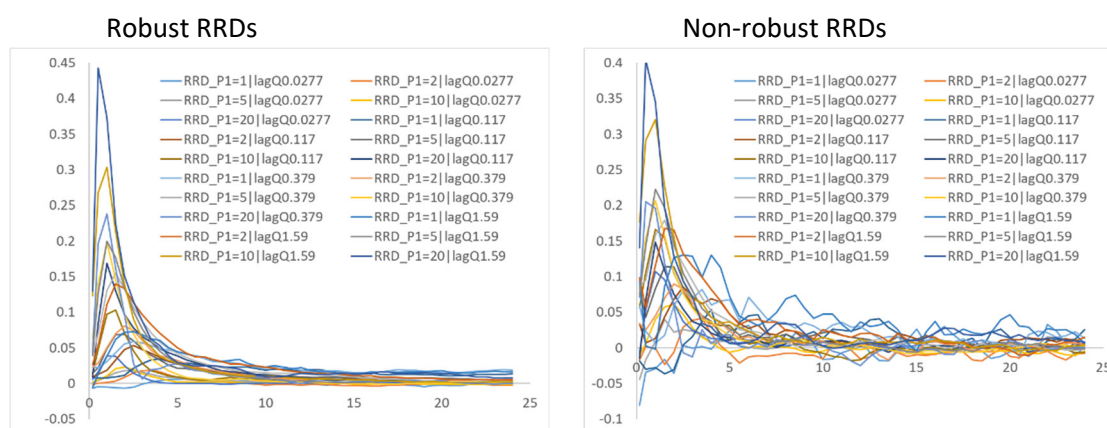
```
lagQ_split <- list(
  crit = list(q) ,
  crit_label = "lagQ" ,
  crit_lag = 2,
  breakpts = list(c(50, 80, 95)) ,
  pct_breakpts = TRUE ,
  thresh = 0 ,
  by_bin = TRUE)
```

```
zz <- ERRA(p=p, q=q, m=48, split_params=lagQ_split, Qfilter=snowfree,
knots=c(1, 2, 5, 10, 20), pct_knots=FALSE, agg=3, dt=1/6)
```

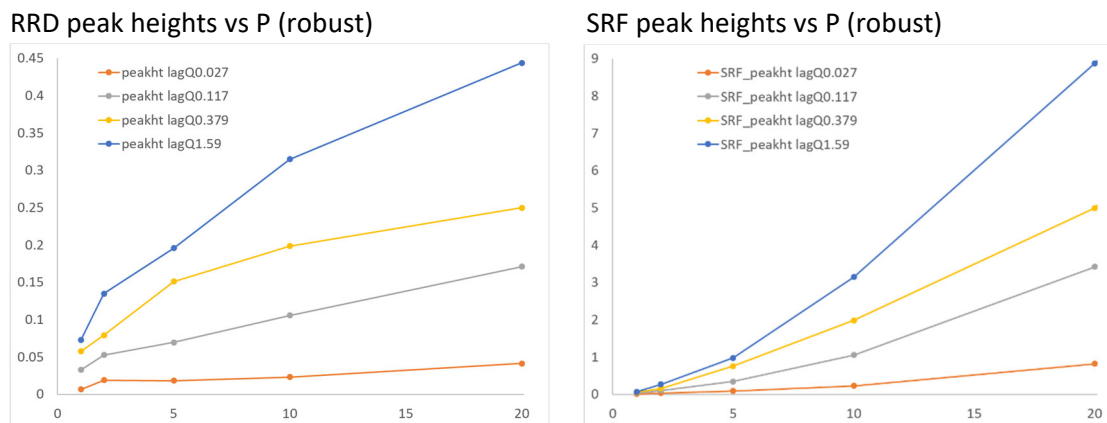
and for the robust version:

```
zz <- ERRA(p=p, q=q, m=48, split_params=lagQ_split, Qfilter=snowfree,
knots=c(1, 2, 5, 10, 20), pct_knots=FALSE, robust=TRUE, agg=3, dt=1/6)
```

The plots below show the robust and non-robust RRDs resulting from these calls to ERRA. The robust and non-robust RRDs exhibit similar patterns, but the robust RRDs are much less noisy.

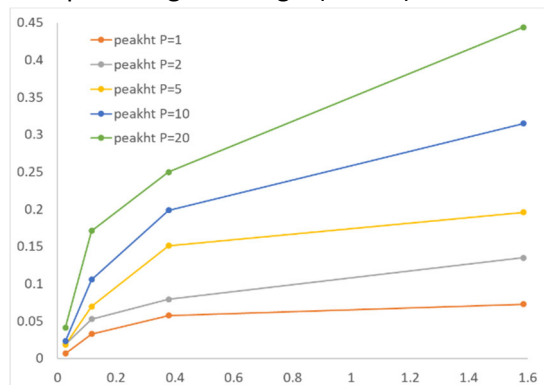


The plots below show the peak heights of the RRDs and SRFs as functions of precipitation intensity (up to 20 mm/h), with separate curves for different ranges of antecedent wetness. The sensitivity of the system to both antecedent wetness and precipitation intensity is visually obvious here.

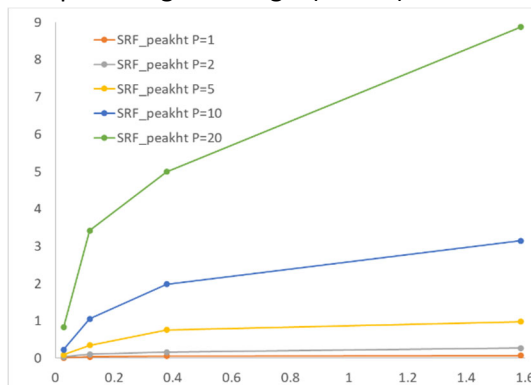


Below we see slices through the same data along the lagQ axis. The plots below show the peak heights of the RRDs and SRFs as functions of lagQ , with separate curves for different knot values of precipitation intensity. Like the plots above, these plots show the sensitivity of the system to both antecedent wetness and precipitation intensity, but this time with an emphasis on antecedent wetness (as reflected in antecedent discharge). Notably, antecedent discharges below about 0.2-0.4 mm/h have a bigger effect (steeper slope) than those above about 0.2-0.4 mm/h.

RRD peak heights vs lagQ (robust)

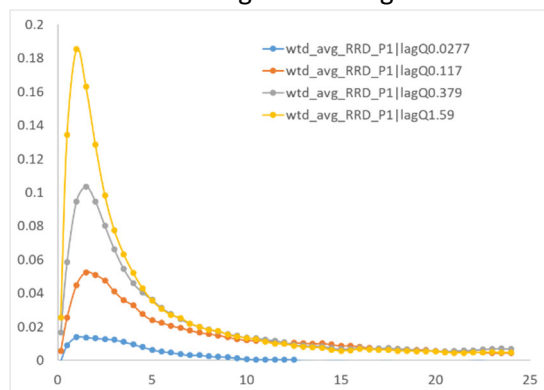


SRF peak heights vs lagQ (robust)

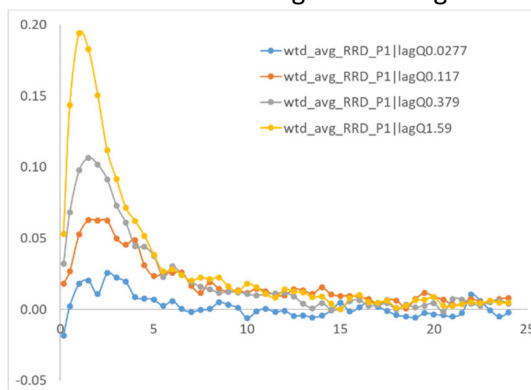


We can use the weighted average RRDs to quantify the average effect of variations in antecedent wetness, averaged over all precipitation inputs. As the figures below show, the robust and non-robust weighted average RRDs are generally similar, but the non-robust ones are noisier.

Robust weighted average RRDs

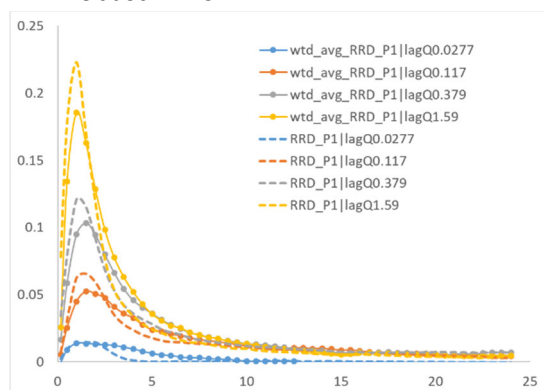


Non-robust weighted average RRDs

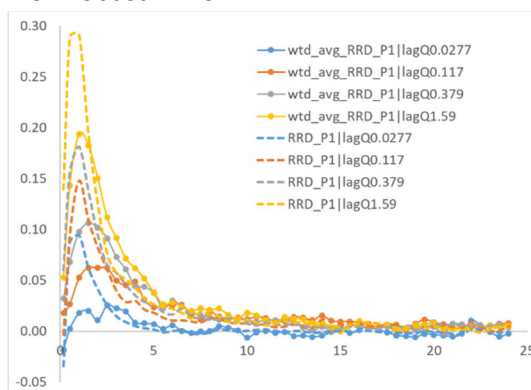


We can also ask how these weighted average RRDs compare to the RRDs that we would obtain if we ignored runoff's nonlinear dependence on precipitation intensity (as we did in sections 3-5 above). In the figures below, the weighted average RRDs for each lagQ range are shown by solid lines with dots (identically to the figures above) and the dashed lines show "regular" RRDs for the same lagQ ranges, that do not account for nonlinear dependence on precipitation intensity.

Robust RRDs



Non-robust RRDs



From the figure above it is clear that the robust RRDs that ignore nonlinear dependence (the dashed lines in the left-hand plot) are reasonable approximations of the weighted average RRDs. By contrast, the non-robust RRDs that ignore nonlinear dependence (the dashed lines in the right-hand plot) have peaks that are much too high and much too early, compared to the weighted average RRDs. These results are consistent with section 5.3 of K2022, which showed that ignoring the nonlinear dependence of outputs on inputs can lead to substantial biases in impulse response functions (K2022 did not examine the effects of the robust estimation algorithm because it had not been implemented at that time).

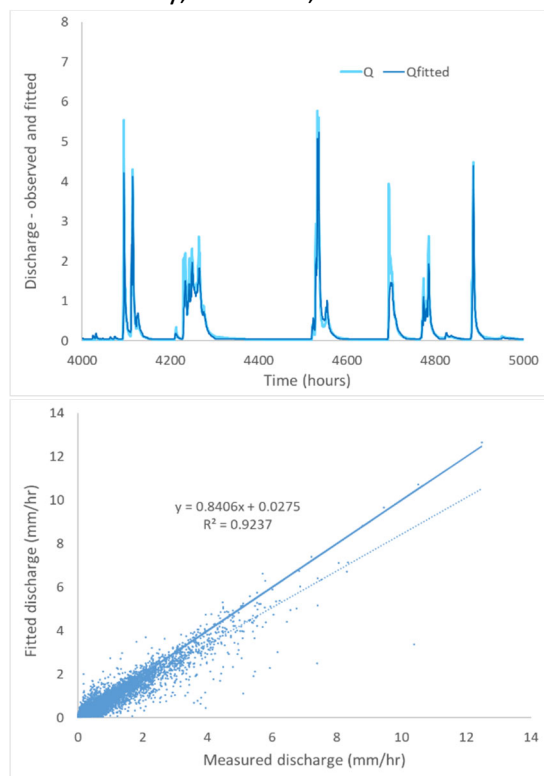
The analyses in this section lead to the following general advice:

- even if you don't care about the nonlinear dependence of runoff on precipitation intensity, you should apply the broken-stick method anyhow (do not set `knots=NULL`), and use the weighted average RRD as the best estimate of the system's average behavior,
- and you should generally use robust estimation, particularly if `knots=NULL`.

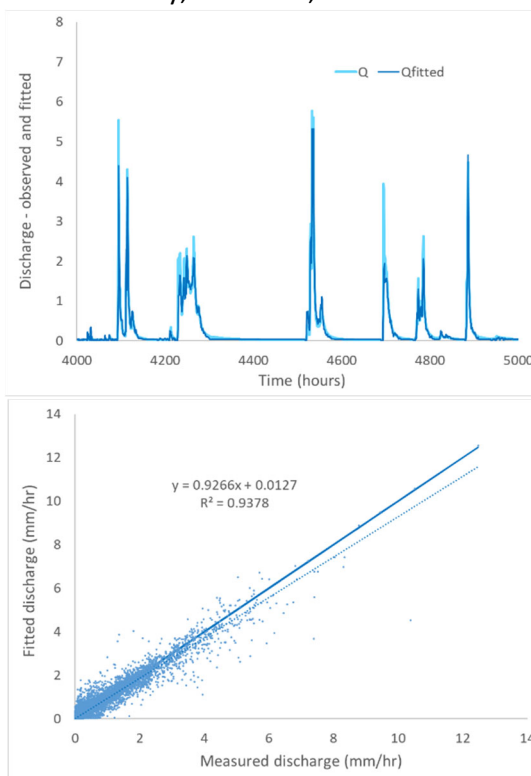
Note, however, that this general advice may change as we do more benchmark testing!!

Lastly, again with the proviso that the objective of ERRA is not to predict hydrographs, we can compare the fitted discharges from this nonlinear, nonstationary regression model to the observed discharges using the `Qcomp` data table. The figures below show that the fitted hydrographs reproduce most of the features of the measured hydrographs, and show that the fitted and measured discharges are strongly correlated (the solid lines are 1:1 lines; the dotted lines are regression lines). We need to be careful to use the word *fitted* rather than *predicted* here, since these are not a priori predictions. Split-sample cross-validation tests would be needed to determine whether this nonlinear, nonstationary model adequately predict discharges that are outside of the calibration data set, before any such model is used to make predictions.

Nonstationary, nonlinear, robust RRD

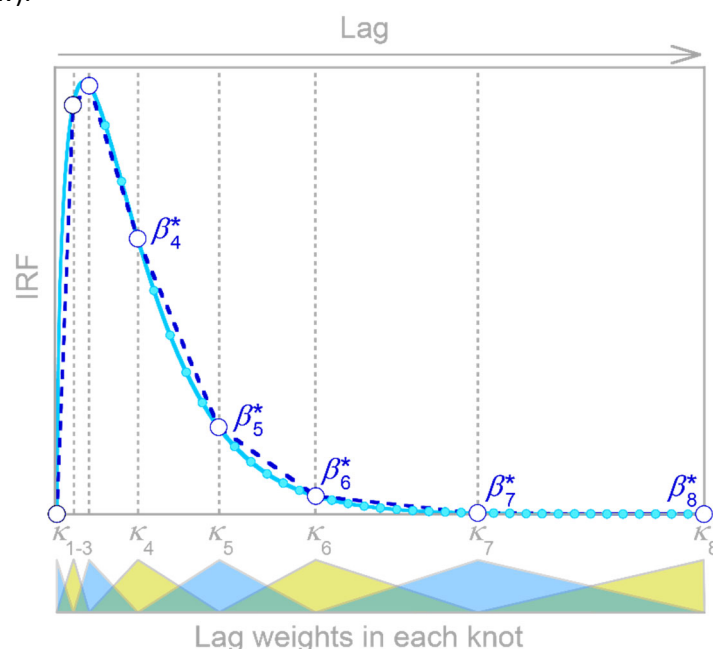


Nonstationary, nonlinear, non-robust RRD



7. Piecewise-linear RRDs for multiscale runoff responses

Runoff responses often span a range of timescales, with a strong, sharp peak at short lags, but a long tail extending to very long lags. In theory, of course, such multi-scale behavior could be captured by evaluating the RRD for a very large number of very closely spaced lags (for example, 1440 10-minute lags to capture responses out to timescales of 10 days). In practice, this can result in a very noisy tail, because the delayed runoff response is too weak to adequately constrain the very large number of individual lag coefficients. Wouldn't it make sense to somehow estimate the RRD for closely spaced lags for short lag times, and wider-spaced lags for long lag times? Indeed it would make sense, and the ERRA script provides a way to do it. As explained in Section 6 of K2022, ERRA can make a piecewise-linear approximation to the RRD, as defined by knot points that are geometrically distributed in lag time with close spacing at short lags, and increasingly wide spacing at longer lags (see the figure below).



Piecewise linear approximation (dashed dark blue line) to an impulse response function (light blue line), with 40 conventional IRF coefficients (light blue dots) replaced by values at 8 knots (open circles). The overlapping yellow and blue triangles depict the relative influence of each lag on the even and odd numbered knots, respectively (see Equations 51-52 of K2022).

Consider the following possible application. Let's say that we want to examine how the rainfall-runoff relationship at Erlenbach varies depending on the temperature when precipitation falls. Let's say that we want to examine three temperature ranges: less than 0 degrees (snow-dominated), 0 to 4 degrees (snow-rain transition), and above 4 degrees (rain-dominated). We can split the analysis by temperature using the following instructions:

```
T <- dat$airtemp_degC

airtemp <- list(
  crit = list(T) ,
```

```
crit_label = "T" ,
crit_lag = 0,
breakpts = list(c(0, 4)) ,
pct_breakpts = FALSE ,
thresh = -999 ,
by_bin = FALSE)
```

In addition, we need to account for the nonlinearity in Erlenbach's response to precipitation (even if we only want an accurate picture of the average response), so let's say that we want use a broken-stick model with three interior knots at the 80th, 95th, and 99th percentiles of the nonzero precipitation. And let's further assume that we want to monitor the rainfall-runoff relationship at Erlenbach out to lags of 30 days, so that we can check for delayed responses to snowfall or rain/snow mixtures. Even if we aggregate to time steps of 30 minutes (the largest time steps that can still resolve the peak response), 30 days is 1440 lags. The function call will look like this:

```
zz <- ERRA(p=p, q=q, m=1440, h=4, split_params=airtemp, xknots=c(80, 95,
99), pct_xknots=TRUE, robust=TRUE, agg=3, dt=1/144)
```

where we have set $dt=1/144$ so that the answers are in units of days (and the RRD is in units of 1/days), to make the results more intuitive. Note also that we have specified $h=4$ because we know that this calculation will be slow, so we don't want to iteratively search for an optimal value of h . Instead, our strategy will be to pick a value of h that we think will be generous, and then verify that choice by looking at the fitted AR coefficients and the residual PACF.

Why do we know that this calculation will be slow? Because our design matrix will have about $(m+h+1)*(n(xknots)+1)*(n(breakpts)+1) = 1445*4*3 = 17,340$ columns, and about 150,000 rows. That's a huge matrix. And we are running this with robust estimation (trust me, we'll need it, because otherwise the results will be horribly noisy), so there will be multiple iterations needed to establish the robustness weights.

Here's what we get if we run the instructions above on a pretty fast (2019 vintage) laptop:

```
initialization done...: 0.174
robustness iteration 1 complete at time 1503.075 s2e= 0.05575717 max e= 9.245 rw chg= 1
robustness iteration 2 complete at time 2973.224 s2e= 0.01108919 max e= 9.951 rw chg= 0.387
robustness iteration 3 complete at time 4363.808 s2e= 0.00639378 max e= 10.095 rw chg= 0.214
robustness iteration 4 complete at time 6388.549 s2e= 0.00489966 max e= 10.152 rw chg= 0.138
robustness iteration 5 complete at time 7899.748 s2e= 0.00420985 max e= 10.181 rw chg= 0.098
IRF finished...: 9875.222 e...: 9545.853
minimum column count of nonzero x values = 46 (0% of maximum count, and 0% of rows with nonzero weight)
AR coefficients (phi) = 1.284269 -0.4724858 0.1007566 -0.0001704115
residual PACF (first 5 terms) = 0 -0.0024 0.007 -0.017 -0.0097
return from IRF...: 9907.23
ERRA_finished...: 9910.089
```

So this took about 10,000 seconds, or nearly three hours. Our AR correction looks good, because the residual PACF values are small, and because the AR coefficients die away to nearly zero (meaning that adding more AR coefficients wouldn't help).

Now let's try the same thing with the piecewise linear approximation, using 30 knots (at lag 0 and a geometrical distribution of lags between 1 and 1440, inclusive). To do that, we just add the option `nk=30` to the function call above, where `nk` stands for "number of knots":

```
zz <- ERRA(p=p, q=q, m=1440, nk=30, h=4, split_params=airtemp, xknots=c(80,
95, 99), pct_xknots=TRUE, robust=TRUE, agg=3, dt=1/144)
```

Here's what we get when we run this on the same laptop as before:

initialization done...: 0.292

robustness iteration 1 complete at time 81.578 s2e= 0.05280498 max e= 11.347 rw chg= 0.947

robustness iteration 2 complete at time 95.293 s2e= 0.00770765 max e= 11.36 rw chg= 0.351

robustness iteration 3 complete at time 105.629 s2e= 0.00482555 max e= 11.369 rw chg= 0.205

robustness iteration 4 complete at time 116.347 s2e= 0.00381603 max e= 11.374 rw chg= 0.135

robustness iteration 5 complete at time 126.112 s2e= 0.00329951 max e= 11.377 rw chg= 0.097

IRF finished...: 259.305

minimum column count of nonzero x values = 46 (0% of maximum count, and 0% of rows with nonzero weight)

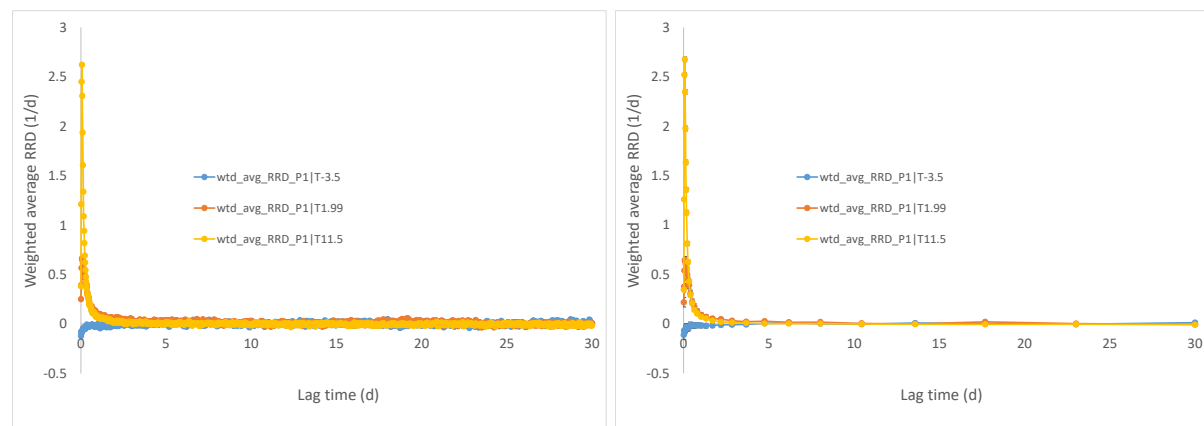
AR coefficients (phi) = 1.236757 -0.4199786 0.08044835 0.006144535

residual PACF (first 5 terms) = -0.0249 0.0215 0.0442 0.0633 0.0593

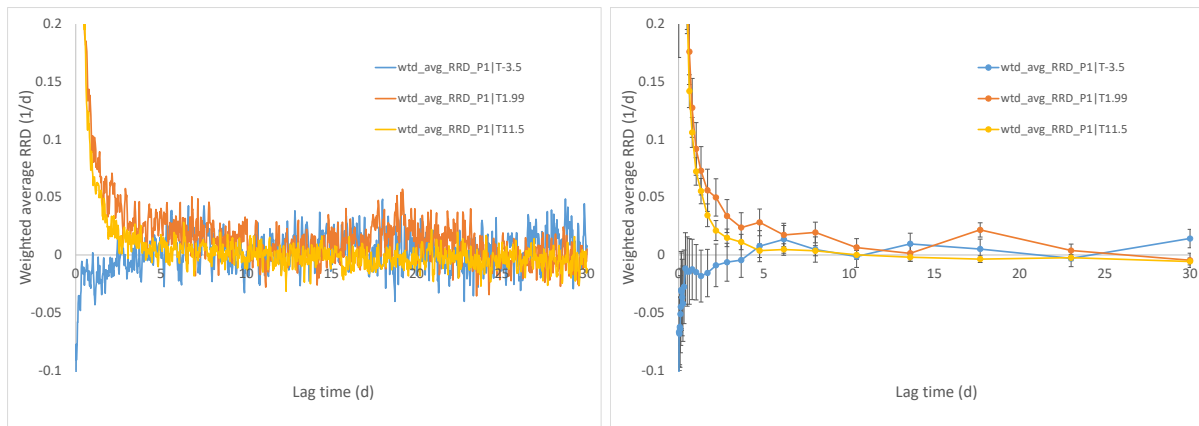
return from IRF...: 262.068

ERRA_finished...: 262.162

So this was over 30 times faster, taking about 4.5 minutes instead of nearly three hours. The results are nearly identical, as the side-by-side comparison (of the weighted RRDs for each of the temperature ranges) shows:



There is, however, an important difference, which is that at lags between 10 and 30 days, the uncertainties (standard errors) in the piecewise-linear approximation are roughly a factor of 5 smaller than those in the "regular" RRD. (At very short lag times, where the piecewise linear approximation is evaluated at each lag, its uncertainties are similar to those in the regular RRD.) This more clearly reveals the differences in the long-lag response to precipitation falling at different temperatures (error bars are not shown on the left-hand plot, and would make it look even fuzzier):



In particular, in the piecewise-linear RRD, one can see that at most lags longer than about 5-10 days, the response to precipitation falling at below-zero temperatures (blue) or 0-4 degrees (orange) is often detectably elevated above the response to precipitation falling above 4 degrees. Whether this is due to the difference in precipitation phase, or due to other differences that are correlated with temperature (such as vapor pressure deficit, season, or vegetation activity) is of course an open question.

Users should note that, as explained in K2022, the AR noise correction can misbehave when the piecewise linear approximation is used. In particular, the automatic AR selection method may not converge, because the residual PACF may never become small enough. Thus a more pragmatic approach to selecting the AR order parameter h is (as shown above) to make a generous guess and then verify either that the residual PACF is well behaved, or that the AR coefficients are dying away to nearly zero (so that further increases in h would not help).